LAB 10 – CODE REVIEW AND QUALITY

Using AI to Improve Code Quality and Readability

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task 1 – Syntax and Logic Errors

Question

Use AI to identify and fix syntax and logic errors in the given Python script that calculates a student's average score.

Prompt:

"Review this Python code, fix syntax and logic errors, keep the same purpose, and briefly explain every correction."

Code

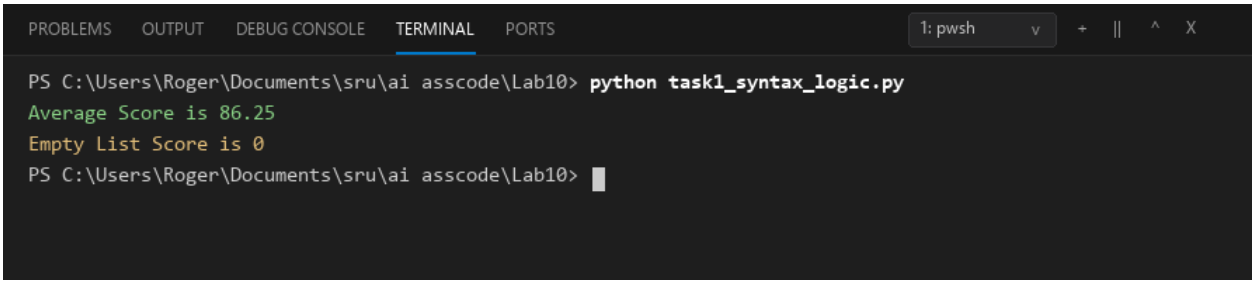
```
def calc_average(marks):
    """Return the arithmetic mean of marks, or 0 for an empty list."""
    if not marks:
        return 0

    total = 0
    for mark in marks:
        total += mark

    average = total / len(marks)
    return average


if __name__ == "__main__":
    marks = [85, 90, 78, 92]
    print("Average Score is", calc_average(marks))
    print("Empty List Score is", calc_average([]))
```

Output:



Justification

The variable name 'avrage' was misspelled; it was corrected to 'average'. A closing parenthesis was added to the print statement. The function now handles an empty list to avoid division by zero. The average of 85, 90, 78, and 92 is 86.25.

Task 2 – PEP 8 Compliance

Question

Refactor the rectangle-area program to follow PEP 8 style guidelines.

Prompt:

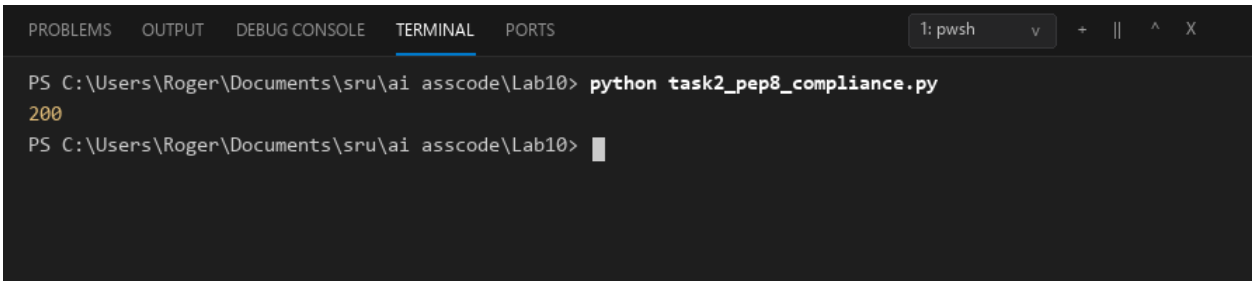
"Rewrite this Python code using PEP 8 naming, spacing, and formatting. Keep the calculation and result unchanged."

Corrected Code

```
def area_of_rectangle(length, breadth):
    """Return the area of a rectangle."""
    return length * breadth


if __name__ == "__main__":
    print(area_of_rectangle(10, 20))
```

Output:



Justification

The function name is descriptive, parameters use lowercase words, spaces are placed around operators, and the function body is formatted across multiple lines. A docstring explains the function. The logic remains length * breadth, so the output is still 200.

Task 3 – Readability Enhancement

Question

Improve readability without changing the logic. Use descriptive variable names, inline comments, and clear formatting.

Prompt:

"Improve the readability of this code without changing its calculation. Use meaningful names, clear formatting, and a helpful comment."

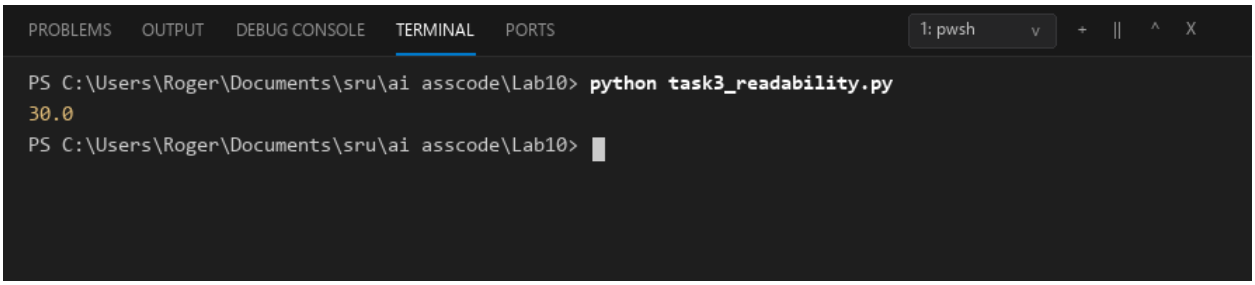
Code

```
def calculate_percentage(value, percentage):
    """Calculate a percentage of a given value."""
    # Multiply the value by the percentage and divide by 100.
    return value * percentage / 100


if __name__ == "__main__":
    total_value = 200
    percentage_rate = 15

    result = calculate_percentage(total_value, percentage_rate)
    print(result)
```

Output:



Justification

The unclear function name 'c' was changed to 'calculate_percentage', and short variable names were replaced with descriptive names. A docstring and inline comment explain the calculation. The formula and values remain unchanged, so the result is 30.0.

Task 4 – Refactoring for Maintainability

Question

Refactor repetitive welcome messages into a reusable function.

Prompt:

"Refactor the repeated welcome print statements into a reusable function and loop through the student list. Keep the same messages."

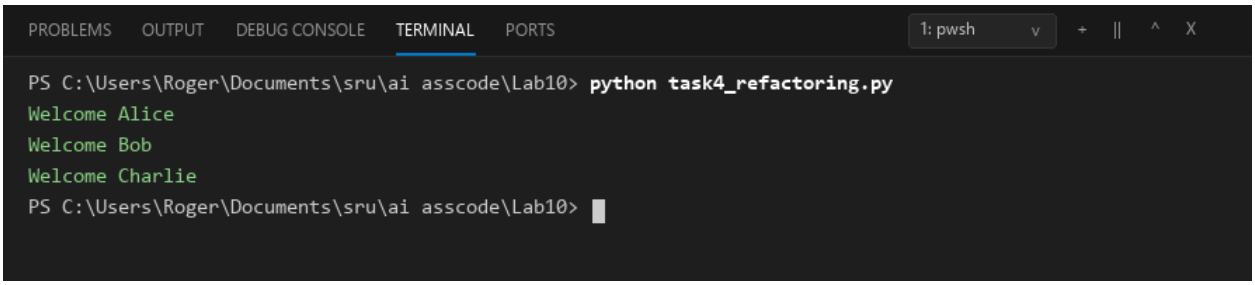
Code

```
def welcome_student(student_name):
    """Print a welcome message for one student."""
    print("Welcome", student_name)


if __name__ == "__main__":
    students = ["Alice", "Bob", "Charlie"]

    for student in students:
        welcome_student(student)
```

Output:



Justification

The repeated print statements were moved into the reusable welcome_student() function. A loop processes every name in the list, reducing duplication and making the program easier to maintain. If more students are added, no extra print statements are required.

Task 5 – Performance Optimization

Question

Optimize the program that calculates squares of numbers from 1 to 999,999.

Prompt:

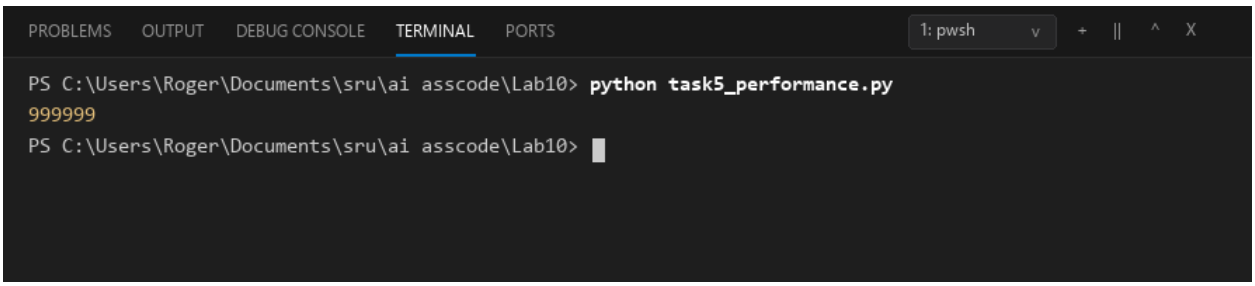
"Optimize this Python code for speed and simplicity. Avoid creating an unnecessary intermediate list of numbers, preserve the final count, and explain the change."

Code

```
def generate_squares_count(limit=1_000_000):
    """Generate squares from 1 up to limit - 1 and return the count."""
    squares = [number ** 2 for number in range(1, limit)]
    return len(squares)


if __name__ == "__main__":
    print(generate_squares_count())
```

Output:



Justification

The separate nums list and append loop were removed. A list comprehension creates the square values in one concise expression and avoids storing a separate list of the original numbers. The program still stores the squares because the task asks to build a squares list and print its length. If only the count were required, len(range(1, 1_000_000)) would be even more memory-efficient.

Task 6 – Complexity Reduction

Question

Simplify the nested if-else logic used to assign a grade based on a score.

Prompt:

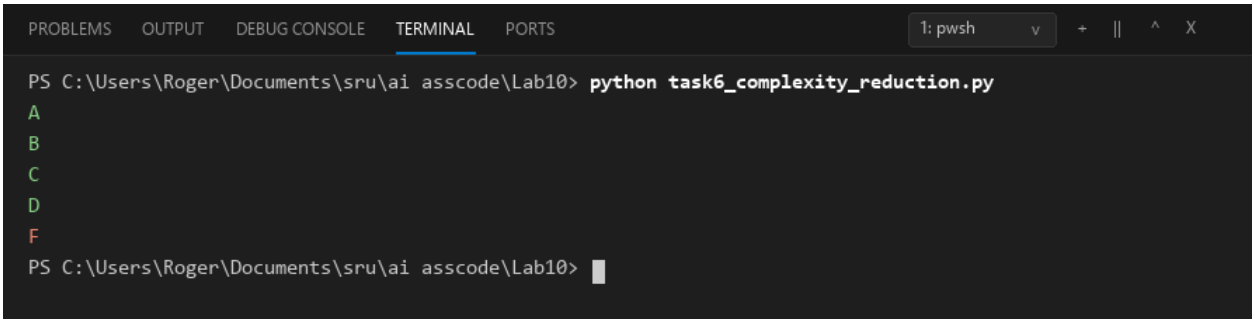
"Simplify the nested grade function using elif. Preserve the existing grade boundaries and return values, and include a few test cases."

Code

```
def grade(score):
    """Return a letter grade based on the score."""
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"


if __name__ == "__main__":
    print(grade(95))
    print(grade(82))
    print(grade(74))
    print(grade(65))
    print(grade(40))
```

Output:



Justification

The nested if-else statements were replaced with a clear if-elif-else chain. Since conditions are checked from highest to lowest score, the first matching condition returns the correct grade. The score thresholds and grade labels are unchanged, while readability and maintainability are improved.